

LC 664: Strange Printer (Cisco, Google)

Dynamic Programming approach and Notes

© Authored by: Arpit Raj, LNMIIT Jaipur | Date: September 01, 2026

Problem Statement

Printer can print a sequence of the same character each time. Return the minimum number of turns the printer needs to print a string.

Examples

Eg: aaabbb

Hence: $1 + 1 = 2$

Eg: aba

Print **aaa** first and then print **b** at the second place.

So: $1 + 1 = 2$

Eg: aadya

Print **aaaaa** first, then **d**, and then **y**.

So: $1 + 1 + 1 = 3$

Explanation: If print naively then 4 turns (aa, d, y, a). But those 3 'a' can be printed once, hence 3 turns.

Dynamic Programming Formulation

Let **dp[1][r]** = minimum turns to print **s[1...r]**.

Base Case

To print a single character need a single turn. Hence:

```
dp[i][i] = 1
```

Option 1

For range `[1][1+1.....r]` : Isko pehle print karo, pher `s[1+1.....r]` ko print karo.

```
dp[1][r] = 1 + dp[1+1][r]
```

Option 2

When `s[1] == s[j]`, i.e., we found another same character in range `[1.....j-1]` `[j.....r]`. To `s[1]` aur `s[j]` ko same turn me print kar do.

```
dp[1][r] = dp[1+1][j-1] + dp[j][r]
```

Example context for Option 2: Eg `aba`, `s[1] == s[j]`.

Hence `dp[1][r] = dp[1+1][j-1] + dp[j][r]`.

Here `dp[1+1][j-1]` handles `b`, and `dp[j][r]` handles `a`.

Code Implementation

```
class Solution {
public:
    int strangePrinter(string s) {
        int n=s.size();

        //dp[l][r]
        vector<vector<int>> dp(n,vector<int>(n,INT_MAX));

        for(int i=0;i<n;i++){
            dp[i][i]=1;
        }

        for(int len=2;len<=n;len++){
            for(int l=0;l+len-1<n;l++){
                int r=l+len-1;

                //opt1
                //dp[l][l+1....r]
                dp[l][r]=1+dp[l+1][r];

                //opt2
                //dp[l....j-1][j....r]
                //aadyarastogi
                //l    j        r
                //print l to j together
                //hence bas beech walo ka cost
                //[j...r] me starting waale a bhi count ho jaayege
                //so dp[l+1][j-1]+dp[j][r]
                //l me jo same tha uska cost dp[j][r] me include ho jaayega
                //l ke liye koi alag cost nhi

                for(int j=l+1;j<=r;j++){
                    if(s[l]==s[j]){
                        int curr=dp[j][r];
                        if(j>l+1)
                            curr+=dp[l+1][j-1];

                        dp[l][r]=min(dp[l][r],curr);
                    }
                }
            }
        }

        return dp[0][n-1];
    }
};
```